Day 4 Lab Kickoff

Four parts, one scenario — Sequential's ceiling, the graph that fixes it, bound the loop, then measure Group Chat against both

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/workflows/ · +1 in notes

What you'll build

- The same three roles — Planner, Retriever, Critic:
  - Part A — SequentialBuilder's one-line shortcut, and the ceiling it runs into
  - Part B1 — a custom WorkflowBuilder graph with a conditional edge that fixes it
  - Part B2 — bound that loop, test-driven: a failing spec is the requirement, not a TODO comment
  - Part C — the same roles a third way, GroupChatBuilder, then measure all three constructions against one golden set
- **Optional stretch** — a Ticket agent that files a real Azure DevOps work item (Day 3's MCP path) when the Critic flags a documentation gap

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/workflows/

Part A — Sequential, and its ceiling

- SequentialBuilder(participants=[planner, retriever, critic]).build() — the three roles, wired in one line
- Run 1 (a single-document question): Planner, Retriever, Critic each run once, in order — clean
- Run 2 (a question spanning two documents): the Critic says approved=false with a precise reason — and the workflow ends anyway
- The ceiling: Sequential is forward-only — there's no edge from the Critic back to the Planner, so a correct rejection has nowhere to go. Part B1 fixes exactly this.

Source: https://learn.microsoft.com/en-us/agent-framework/workflows/orchestrations/sequential

Part B1 — The graph that fixes it

```
# Part A's shortcut — the ceiling this fixes
SequentialBuilder(participants=[planner, retriever, critic]).build()

# Part B1 — a custom graph with a loop-back edge
builder = WorkflowBuilder(start_executor=planner)
builder.add_edge(planner, retriever)
builder.add_edge(retriever, critic)
builder.add_edge(critic, planner, condition=lambda result: not result.approved)
```

Same three roles, same shape — one new edge. When the Critic doesn't approve, the graph now has somewhere to send that verdict: back to the Planner.

Source: https://learn.microsoft.com/en-us/agent-framework/workflows/orchestrations/sequential · +1 in notes

Part C — Group Chat, and bound it too

1 Rebuild

same three roles as plain participants, now with GroupChatBuilder(participants=[...], orchestrator_agent=..., termination_condition=...)

2 Orchestrator picks

an LLM agent reads the shared conversation and picks who speaks next; no condition function to write

3 Shared context

the Planner already sees the Critic's rejection when re-selected, because the whole conversation is shared — no manual to_revision adapter needed

4 Bound it too

termination_condition is required for the same reason Part B2's was: stop once there have been 8 assistant turns, a reasonable ceiling for three roles and up to two revision passes

Source: https://learn.microsoft.com/en-us/agent-framework/workflows/orchestrations/group-chat

Part B2 — Bound the loop

- Part B1's gate loops whenever approved is false — for a Critic that never approves (a genuinely ungroundable question), that's forever
- Test-driven, not authored from scratch: tests/test_guardrail.py ships failing — it IS the specification; edit RevisionGate.decide in workflow_nodes.py until all three tests pass
- The bound: stop at MAX_REVISIONS (2 revisions), stop gracefully — a low-confidence Answer with capped=True, never an exception — and a Critic that DOES approve still short-circuits immediately
- Not the same job as WorkflowBuilder(max_iterations=...): that's a whole-graph backstop (defaults to 100 supersteps, and simply stops with no answer); this is a domain rule that ends the run deliberately, with an answer a caller can use. Keep both.

Source: https://learn.microsoft.com/en-us/agent-framework/agents/looping

Part C — Evaluate and compare

- Imports one build_*_workflow() function each from part_a_sequential.py, part_b_graph.py, and part_c_group_chat.py — no new orchestration code
- Runs the SAME golden-set slice against all three constructions: Sequential, custom graph, Group Chat
- Reports a comparison table: pass rate (as a range across repetitions), cost per success, and total revisions — side by side for all three
- **Reflect** — which approach fit best, and why — write it down, it's part of Done

Source: https://learn.microsoft.com/en-us/agent-framework/agents/evaluation

Build one golden set, use it for Part C

- Same discipline as Day 3 Part E, at workflow scale: real queries, expected outcomes, not synthetic passes
- The lab's own golden set: 8 cases across 4 branches — small on purpose, since every case runs against all 3 constructions × 3 repetitions (~72 workflow runs already)
- Cover the workflow's real branches: grounded (clean single-document lookup), revision (spans documents, needs the loop), no_retrieval (general knowledge, no tools), and partial (corpus answers half — naming the gap is correct, inventing the rest is a failure)
- Build it once, before Part C — every construction runs against the exact same set, so the comparison means something

Source: https://learn.microsoft.com/en-us/agent-framework/agents/evaluation

Run the trajectory evaluation

- **Task success** — deterministic, local, no judge model: the answer contains what must_mention requires and cites what expected_citations requires
- **Citation accuracy** — did it cite the documents the answer actually needs (and cite NOTHING on a no-retrieval question)?
- **Trajectory** — the ordered tool calls against expected_actions; the same ground truth Foundry's Task Navigation Efficiency evaluator consumes when you add --foundry
- **Cost per success** — total tokens across every agent, divided by the cases that passed; a construction that succeeds cheaply beats one that succeeds expensively at the same pass rate

Source: https://learn.microsoft.com/azure/foundry/concepts/evaluation-evaluators/agent-evaluators

Prerequisites

Day 3 lab complete

a working single agent with memory, streaming, structured outputs, and MCP

Bundled reference docs

included in the repo (labs/day4/python/data/docs/); nothing to provision, no live knowledge base required

A Foundry judge model deployment

optional: only for evaluate.py --foundry's cloud evaluator pass; the core comparison is local and deterministic

(Optional stretch only)

Day 3's Azure DevOps MCP path, for the Ticket agent

Source: https://learn.microsoft.com/en-us/agent-framework/agents/evaluation

Definition of done

Area	Done when
Part A	Both runs complete and print a trace you can read; you can explain why Run 2 ends without fixing itself
Part B1	The graph runs end-to-end and the Mermaid diagram shows the loop-back edge; you can point to the one line that closes the cycle
Part B2	All of tests/test_guardrail.py passes — required, not stretch
Part C	The GroupChatBuilder construction runs end-to-end with a working termination bound; you've run the 3-repetition comparison
Reflection	Which construction you'd actually ship, and why — committed to the repo
Stretch	Ticket agent files a real ADO work item when the Critic flags a gap

Source: https://learn.microsoft.com/en-us/agent-framework/agents/evaluation · +1 in notes

Takeaways

- ✓ You start with the fastest correct orchestration and find its ceiling before you fix it — Part A's SequentialBuilder, then Part B1's custom graph, deliberately in that order.
- ✓ You build the same three roles three different ways — Sequential, a custom graph, Group Chat — and watch the same limitation get fixed two different ways.
- ✓ You build the guardrail Module 6 argued for as a test-driven spec, not an afterthought — Part B2's failing test IS the requirement, required for both loops this lab can trigger.
- ✓ You measure every construction against the same golden set in one dedicated evaluation pass, so the differences you see are real, not noise.

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/workflows/ · +1 in notes